



Saint Columban College
College of Computing Studies
Pagadian City



Bookmarklet

Bookmarklet 



Easy

Web Exploitation

picoCTF 2024

obfuscation

browser_webshell_solvable

browser

AUTHOR: JEFFERY JOHN

Description

Why search for the flag when I can make a bookmarklet to print it for me?

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is:

NOT_RUNNING

Launch Instance

Hints 

1 2 3

A bookmarklet is a bookmark that runs JavaScript instead of loading a webpage.

60,863 users solved



92%
Liked



 picoCTF{FLAG}

Submit
Flag

Author: The Analyst: Hyposelenia

Challenge: Bookmarklet

Category: Web Exploitation

Date: 02/24/26



I. Objective

?

II. Background

A **bookmarklet** is a small piece of JavaScript code saved as a browser bookmark. When clicked, it executes JavaScript directly in the context of the current webpage.

Bookmarklets are commonly used for:

- Quick browser-based tools
- Page manipulation
- Automation
- Security testing

In web exploitation, JavaScript can:

- Access cookies
- Modify page content (DOM)
- Reveal hidden information
- Trigger hidden functionality

This challenge tested knowledge of:

- JavaScript execution
- Browser Developer Tools
- Client-side code analysis

Since the script was provided directly on the page, the main challenge was understanding *how and where to execute it*.

III. Tool Used

- **Web Browser** - Used to access and interact with the challenge website.
- **Browser Developer Tools (Inspect → Console)** – Used to execute JavaScript manually.



- **Online JavaScript compiler (e.g., Programiz)** – Alternative way to run the script.

IV. Methodology

1. Launched the challenge instance.

Its current status is:

NOT_RUNNING

Launch Instance

2. Accessed the provided challenge link.



Welcome to my flag distribution website!

If you're reading this, your browser has
succesfully received the flag.

Here's a bookmarklet for you to try:

```
javascript:(function() {  
  var encryptedFlag =  
  "a0Ab}E-8Ue0m0U&0N400E0EA0CUEi";  
})
```

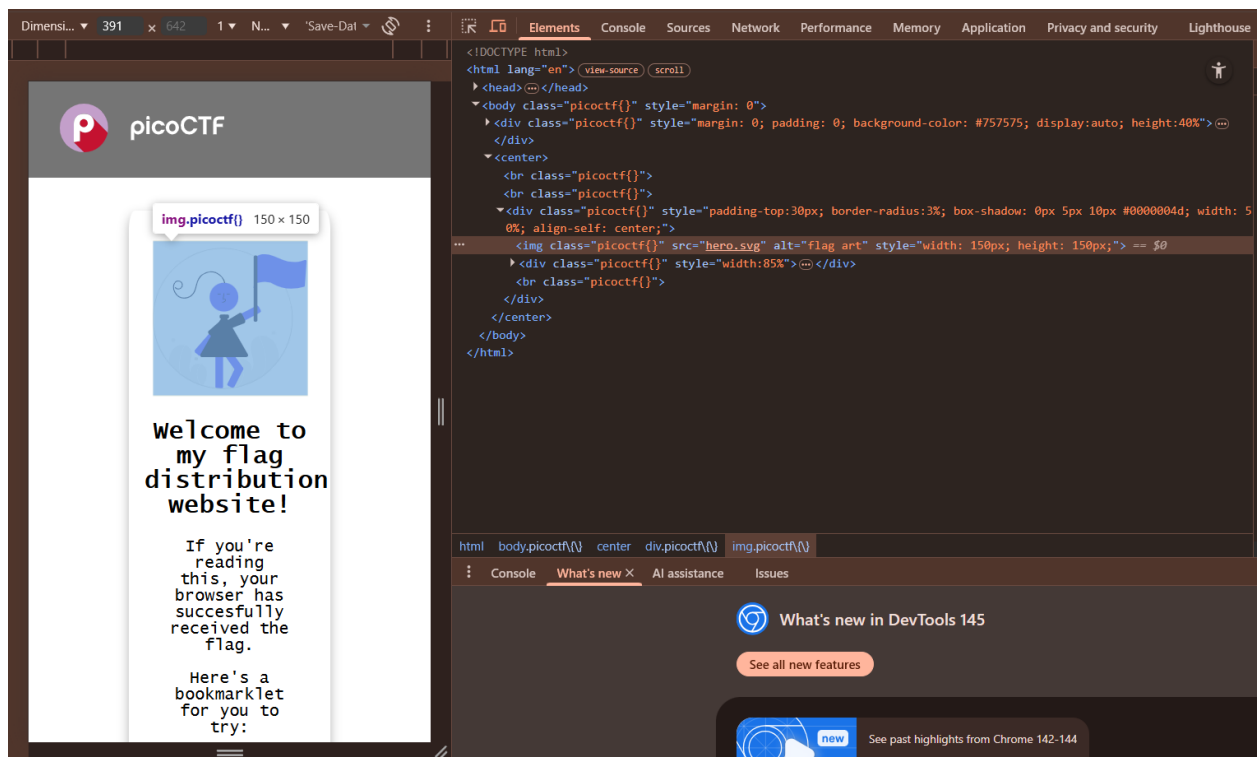
3. Copied the JavaScript code displayed on the page.



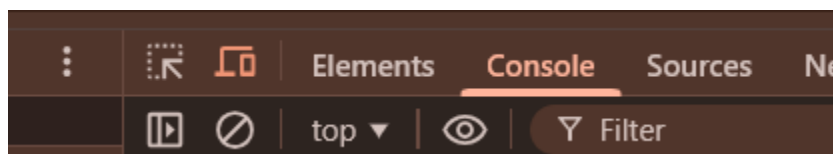
Here's a bookmarklet for you to try:

```
javascript:(function() {  
    var encryptedFlag =  
    "àÒÀP!È-èÜËÖÖÚáÜÑçÖÖËÖÀÒÇÜËí";
```

4. Right-clicked on the page → Clicked **Inspect**.



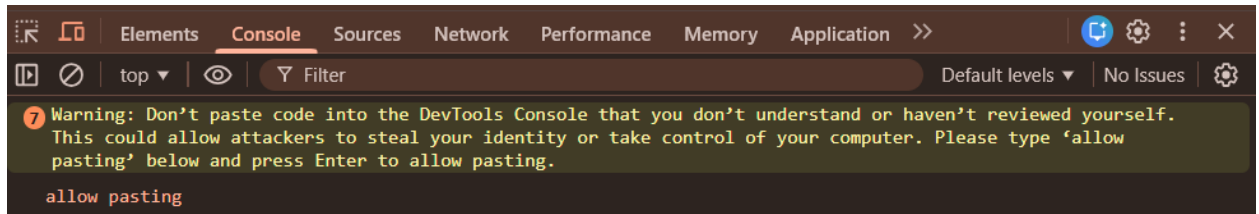
5. Navigated to the **Console** tab.





6. Typed:

allow pasting



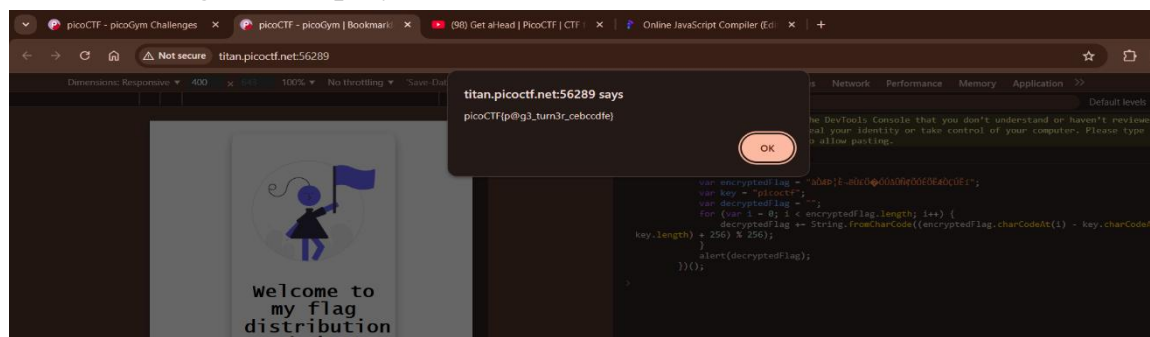
and pressed Enter (**required in some browsers for security reasons**).

7. Pasted the copied JavaScript function into the console.



8. Pressed Enter to execute.

9. The flag was displayed.





Alternative method:

10. Paste the JavaScript code into an online JavaScript compiler and execute it there.

```
main.js
1 javascript:(function() {
2   var encryptedFlag = "àðēþ;E~eUēÖ·ÖÚâÜÑcÖÓÉÖēÖçÜē1";
3   var key = "picoctf";
4   var decryptedFlag = "";
5   for (var i = 0; i < encryptedFlag.length; i++) {
6     decryptedFlag += String.fromCharCode((encryptedFlag.charCodeAt(i) - key
7       .charCodeAt(i % key.length) + 256) % 256);
8   }
9   alert(decryptedFlag);
10 })();
```

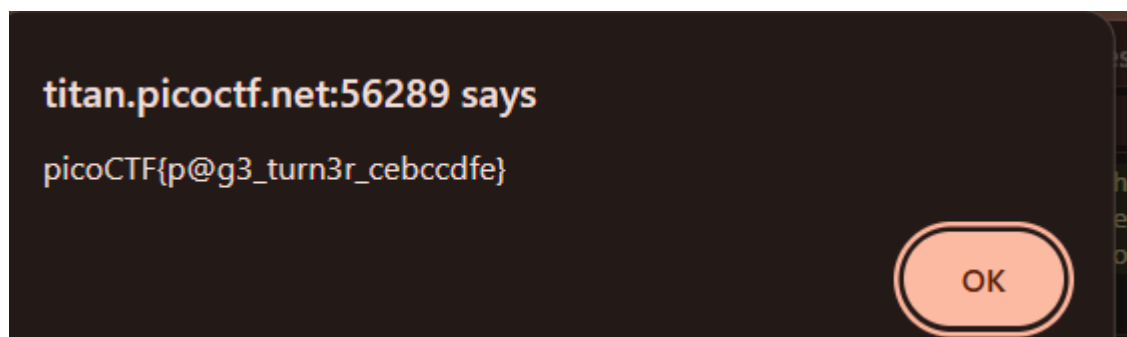
Output

picoCTF{p@g3_turn3r_cebccdfe}

=== Code Execution Successful ===

V. Result

picoCTF{p@g3_turn3r_cebccdfe}



```
Output
picoCTF{p@g3_turn3r_cebccdfe}
=== Code Execution Successful ===
```



VI. Explanation

What is a Bookmarklet?

A bookmarklet is:

javascript: (function(){ ... })();

It is essentially:

- A self-executing JavaScript function
- Run directly in the browser context
- Able to access the current page's DOM and data

Unlike regular bookmarks (which open URLs), bookmarklets execute code.

Why Developer Tools Were Used?

Modern browsers block direct pasting into the console for security reasons to prevent malicious copy-paste attacks.

Typing:

allow pasting

temporarily enables pasting in the console.

The console executes JavaScript in the context of the currently loaded webpage, meaning:

- It can access variables
- It can read cookies
- It can manipulate the page

If the challenge script referenced page data, it needed to be run inside the page context.

What Happened When the Script Ran?

When executed, the JavaScript:

- Decoded or computed hidden data
- Possibly accessed document properties
- Revealed the flag directly in the output

This demonstrates that:

- Client-side code can expose sensitive information



- Anything shipped to the browser can be analyzed
- JavaScript logic can be reversed or executed manually

VII. Conclusion

The **Bookmarklet** challenge reinforces the importance of understanding:

- Client-side JavaScript execution
- Browser Developer Tools
- The risks of exposing logic in front-end code

The challenge showed that:

- If sensitive logic exists in JavaScript, it can be extracted
- Execution context matters
- Developer tools are powerful for both debugging and exploitation

Overall, this was a foundational web exploitation challenge emphasizing browser-side code analysis and execution.

— The Analyst: Hyposelenia